Cryptographie R309
Substitution monoalphabétique, Partie II
Décryptage par recuit simulé

18 septembre 2026

Objectif Dans la partie I, nous avons constaté qu'une analyse fondée uniquement sur la fréquence des lettres ne permet pas de décrypter un message : seules quelques lettres sont correctement identifiées, car beaucoup ont des fréquences trop proches en français. Il faut donc exploiter une caractéristique plus fine de la langue : la fréquence des *bigrammes*, c'est-à-dire des couples de lettres consécutives.

Nous verrons que le choix de la fonction à optimiser importe autant que l'algorithme d'optimisation, et nous découvrirons au passage une limite qu'aucun algorithme ne peut franchir.

Rendu `mono_decypher.py`

Le code devra être réalisé sous `Python`. Les fonctions demandées (en rose dans l'énoncé) devront être sauvegardées dans un fichier nommé `mono_decypher.py`, à déposer sur le Moodle. Le fichier ne doit contenir ni variable globale ni test.

Test `test-mono_decypher.py`

Un script de test est fourni afin de valider votre code au fur et à mesure.

Fichiers fournis `all_grammes.txt` (occurrences des mono-, bi- et trigrammes sur un corpus Wikipédia) ; `message_alice_bob.txt`.

Exercice 1 Quelle fonction faut-il optimiser ?

Dans les séances précédentes, nous avons mesuré la plausibilité d'un message par une distance en norme L^1 entre fréquences observées et fréquences de référence :

$$J_k(m) = \sum_{x \in \mathcal{A}^k} \left| F(m,x) - \overline{F}(x) \right| .$$

Nous allons voir que cette mesure, satisfaisante pour $k = 1$, devient inadaptée pour $k = 2$.

1. Combien existe-t-il de bigrammes sur un alphabet de 26 lettres ? Le message d'Alice et Bob que nous allons attaquer compte 226 caractères ; combien contient-il de bigrammes, en comptant les répétitions ? Combien au maximum de bigrammes *distincts* ?

2. On observe en pratique 113 bigrammes distincts dans ce message. Pour combien de bigrammes x a-t-on $F(m, x) = 0$? Quelle part de la somme définissant $J_2(m)$ ces termes représentent-ils ?

3. En déduire pourquoi J_2 discrimine mal deux clés candidates.

On adopte donc une autre mesure. On modélise le français par une *chaîne de Markov d'ordre* 1 : la probabilité d'un texte $m = m_1 m_2 \cdots m_n$ est approchée par

$$P(m) \simeq \prod_{i=1}^{n-1} P(m_i m_{i+1}),$$

où $P(ab)$ est la fréquence du bigramme ab dans le corpus de référence. On appelle *log-vraisemblance* de m la quantité

$$L(m) = \sum_{i=1}^{n-1} \log P(m_i m_{i+1}). \tag{1}$$

Décrypter revient alors à chercher la permutation σ qui **maximise** $L(\sigma(m))$.

4. Pourquoi passer au logarithme ? On raisonnera sur l'ordre de grandeur de $P(m)$ pour un texte de 226 caractères, sachant qu'un bigramme fréquent a une probabilité de l'ordre de 10^{-2}.

5. Le corpus de référence ne contient que 649 bigrammes sur les 676 possibles. Que vaut $\log P(x)$ pour un bigramme absent, et quelle conséquence cela a-t-il sur (1) ? Proposer une correction. *On parle de lissage.*

6. $L(m)$ est une somme de $n - 1$ termes tous négatifs. Un texte deux fois plus long a-t-il un meilleur ou un moins bon score ? Est-ce gênant pour notre usage ?

Exercice 2 Décryptage par recuit simulé

1. *(Corpus de référence)* Écrire une fonction `load_grammes` qui lit `all_grammes.txt` et renvoie le dictionnaire des fréquences des mono-, bi- et trigrammes.

   Attention : Le fichier mélange les trois longueurs. Il faut normaliser *séparément* par longueur, de sorte que les monogrammes somment à 1, les bigrammes aussi, et les trigrammes aussi.

2. *(Fréquences d'un message)* Écrire une fonction `F` qui prend un message, un alphabet et un entier k (valant 1 par défaut) et renvoie les fréquences des k-grammes *présents* dans le message. Écrire également `J`, qui reprend la fonction coût J_k des séances précédentes, afin de pouvoir comparer les deux approches.

3. *(Table des log-probabilités)* Écrire une fonction `log_probas` qui prend les fréquences de référence et l'alphabet et renvoie un dictionnaire associant à *chacun* des 676 bigrammes son logarithme de probabilité, en appliquant le lissage discuté à la question 5 de l'exercice 1.

4. *(Score d'un message)* Écrire une fonction `score` qui prend un message et la table précédente, et renvoie $L(m)$ défini par (1). Calculer le score du message chiffré, puis celui d'un texte français quelconque de même longueur. Commenter l'écart.

Il reste à maximiser ce score sur l'ensemble des 26! permutations. Comme en partie I, on procède par transpositions successives, mais on corrige le défaut principal de la descente : son incapacité à sortir d'un maximum local. L'idée du *recuit simulé* est d'accepter parfois une transposition qui *dégrade* le score, avec une probabilité qui décroît au cours du temps.

Algorithme 1 : recuit simulé

function RECUIT(m, T_0, $T_{\min}$, N)
  $\sigma \leftarrow$ clé aléatoire ; $s \leftarrow \text{SCORE}(\sigma(m))$
  $\sigma^\star \leftarrow \sigma$; $s^\star \leftarrow s$
  for $i = 0$ **to** $N - 1$ **do**
    $T \leftarrow T_0 \, (T_{\min}/T_0)^{i/N}$
    tirer deux lettres $x \neq y$ au hasard ; $\tau \leftarrow (x\ y)$
    $s' \leftarrow \text{SCORE}((\tau\sigma)(m))$
    if $s' > s$ **ou** $\text{alea}(0,1) < e^{(s'-s)/T}$ **then**
      $\sigma \leftarrow \tau\sigma$; $s \leftarrow s'$
      if $s > s^\star$ **then**
        $\sigma^\star \leftarrow \sigma$; $s^\star \leftarrow s$
      end if
    end if
  end for
  return $\sigma^\star, s^\star$
end function

5. *(Recuit simulé)* Implémenter `recuit` conformément à l'algorithme 1. On prendra $T_0 = 10$, $T_{\min} = 0{,}05$ et $N = 12\,000$.
   a) Que se passe-t-il si T est très grand ? Et si $T = 0$? Quel algorithme retrouve-t-on dans ce dernier cas ?
   b) Pourquoi conserve-t-on séparément la meilleure clé rencontrée $\sigma^\star$, plutôt que de renvoyer simplement la clé finale ?

6. *(Décryptage)* Écrire une fonction `decrypte` qui effectue plusieurs exécutions indépendantes de `recuit` et conserve la meilleure. Appliquer au message d'Alice et Bob. De quel personnage célèbre parlent-ils ?

7. *(La limite du modèle)* Le texte obtenu comporte cinq caractères faux.
   a) Les identifier et décrire l'erreur *en termes de permutations*. Rapprocher de la dernière question du TD de la séance précédente.
   b) Comparer le score du texte que vous avez trouvé et celui du texte véritable. Lequel est le meilleur ? Conclure : s'agit-il d'un échec de l'algorithme d'optimisation ?
   c) Compter les occurrences des trois lettres fautives dans le texte véritable. Expliquer.
   d) Proposer deux remèdes de nature différente.

8. *(subsidiaire)* Chiffrer un texte français long, puis appliquer `decrypte` à des préfixes de longueur croissante (150, 250, 400, 600 caractères). Tracer le pourcentage de caractères correctement retrouvés. À partir de quelle longueur le décryptage devient-il exact ?

Quelques méthodes utiles en Python. `math.log` ; `math.exp` ; `random.random` ; `random.sample` ; `dict.get(cle, valeur_par_defaut)` ; échange `a, b = b, a`.